NOVOX

Full Project Architecture & Code Explanation

Flutter + Python FastAPI + MySQL + SQLAlchemy

A developer-facing technical document covering the complete system architecture, folder structure, database design, API flow, Flutter pages, backend modules, authentication, attendance, geofencing, fees, salary, reports, WebSocket monitoring, and planned eSSL integration.

Project	NOVOX EdTech
Frontend	Flutter (Mobile / Web)
Current Web Test Target	Microsoft Edge / Flutter Web
Backend	Python + FastAPI + Uvicorn
Database	MySQL
ORM	SQLAlchemy
Authentication	JWT + role-based access control
Real-time layer	WebSocket
Location	GPS + geofencing
Reporting	PDF / CSV
Biometric integration	eSSL – planned integration layer

Document status: Architecture and code responsibilities are based on the NOVOX flow specification and the project structure/code discussed in this development session. Where an integration is planned but not yet implemented, it is explicitly marked as planned.

1. Document Purpose

This document is intended to be used as the main technical explanation of the NOVOX project. It explains what each major code page/file is responsible for, how data moves through the application, how the database supports each module, and how the Flutter frontend communicates with FastAPI.

Primary objectives

- Explain the complete NOVOX architecture from Flutter UI to FastAPI services and MySQL.
- Explain the responsibility of each important backend and frontend file/page.
- Document authentication, role permissions, attendance, geofencing, breaks, checkout, fees, salary, reports, and live admin monitoring.
- Provide a common reference for future development, debugging, project presentation, and deployment.
- Separate implemented architecture from future integrations such as eSSL biometric attendance.

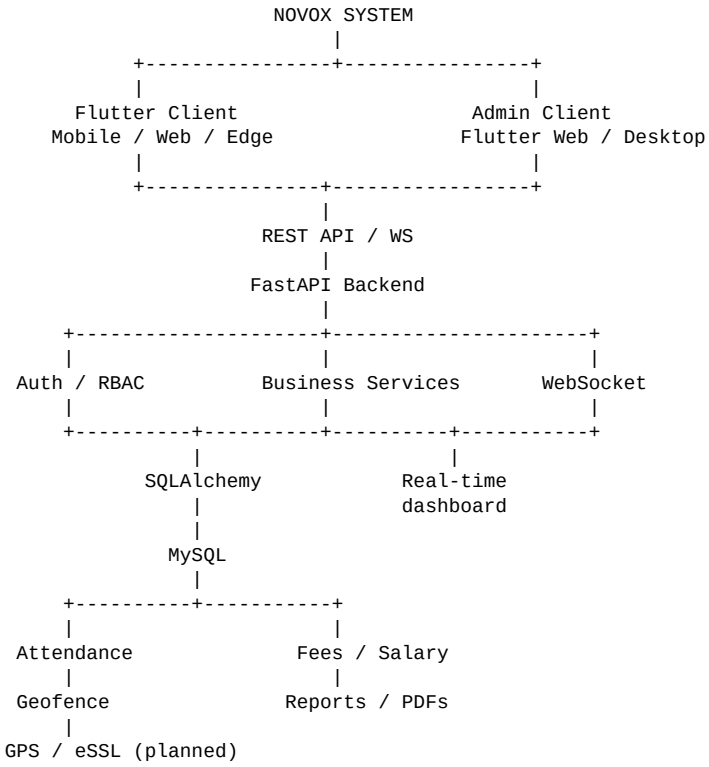
Source specification

The original NOVOX architecture specification describes a Student Portal, Staff Portal, Admin Dashboard, GPS geofence validation, a 10:00 AM attendance cutoff, break/checkout tracking, fee management, staff salary management, JWT role permissions, server-side timestamping, and live database synchronization. The source specifically describes the system as an attendance, geofencing, and management workflow. ■filecite■turn35file0■ ■filecite■turn35file1■

The implementation described in this document follows that functional direction while using the current Flutter + FastAPI + MySQL architecture established during development.

2. Complete NOVOX System Architecture

2.1 High-level architecture



2.2 Main technology responsibilities

Layer	Technology	Responsibility
Presentation	Flutter	Screens, forms, dashboards, navigation, user interaction
Networking	Dio	REST API calls and authorization headers
Storage	flutter_secure_storage	JWT token and local authentication metadata
Location	Geolocator	Device coordinates for geofence attendance
Real-time	web_socket_channel	Admin live attendance updates
API	FastAPI	Routes, validation, authentication, HTTP responses
Application logic	Python services	Attendance, fees, salary, reports, geofence calculations
ORM	SQLAlchemy	Models, queries, relationships, transactions
Database	MySQL	Persistent users, attendance, fees, payments, salaries, geofence settings
PDF/CSV	ReportLab / Python CSV	Receipts, salary slips and report exports

3. End-to-End Data Flow

3.1 Login flow

```
Flutter Login Screen
       |
       | POST /auth/login
       v
FastAPI Auth Route
       |
       v
Find User by email
       |
       v
Verify password hash
       |
       v
Create JWT
       |
       +---- role = student ---> Student Dashboard
       |
       +---- role = staff   ---> Staff Dashboard
       |
       +---- role = admin   ---> Admin Dashboard
```

The Flutter ApiClient attaches the stored JWT as a Bearer Authorization header to protected requests. The backend decodes the token, loads the User record, verifies that the account is active, and applies role restrictions.

3.2 Attendance flow

```
User requests Check-In
       |
       v
Flutter obtains GPS coordinates
       |
       v
POST /attendance/check-in
       |
       v
FastAPI checks today's attendance
       |
       v
Load active geofence settings
       |
       v
Calculate distance from campus
       |
       +---- Outside ---> Reject / Out of Campus
       |
       +---- Inside ----> Server timestamp
                                |
                                +-- <= 10:00 ---> PRESENT / ON TIME
                                |
                                +-- > 10:00 ----> PRESENT / LATE
                                |
                                v
                          MySQL Attendance
```

The source specification defines the geofence rule as: coordinates are compared against the institution perimeter; inside enables check-in, outside locks it; the 10:00 AM threshold determines PRESENT versus LATE ENTRY. ■filecite■turn35file1■

3.3 Admin live monitoring

```
Student/Staff action
       |
       v
FastAPI saves attendance
       |
       +---- MySQL
       |
       +---- WebSocket broadcast
                     |
                     v
             Admin Dashboard
```

```
        |
Live attendance UI
```

4. Complete Project Folder Structure

```
NOVOX/
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   │
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── security.py
│   │   │   ├── dependencies.py
│   │   │   ├── timezone.py
│   │   │   ├── websocket_manager.py
│   │   │
│   │   ├── database/
│   │   │   ├── connection.py
│   │   │   ├── base.py
│   │   │
│   │   ├── models/
│   │   │   ├── user.py
│   │   │   ├── student.py
│   │   │   ├── staff.py
│   │   │   ├── attendance.py
│   │   │   ├── geofence.py
│   │   │   ├── fee.py
│   │   │   ├── salary.py
│   │   │
│   │   ├── schemas/
│   │   ├── services/
│   │   │   ├── attendance_service.py
│   │   │   ├── fee_service.py
│   │   │   ├── salary_service.py
│   │   │   ├── report_service.py
│   │   ├── routes/
│   │       ├── auth.py
│   │       ├── attendance.py
│   │       ├── student.py
│   │       ├── staff.py
│   │       ├── admin.py
│   │       ├── fees.py
│   │       ├── salary.py
│   │       ├── reports.py
│   │
│   ├── .env
│   ├── requirements.txt
│   ├── run.py
│   ├── create_test_users.py
│
├── mobile/
    ├── novox_app/
        ├── lib/
        │   ├── main.dart
        │   ├── core/
        │   ├── models/
        │   ├── services/
        │   ├── providers/
        │   ├── screens/
        │   │   ├── auth/
        │   │   ├── student/
        │   │   ├── staff/
        │   │   ├── admin/
        │   ├── widgets/
        ├── pubspec.yaml
        ├── android/
```

The exact file list may grow as modules are separated further. The important architectural rule is that UI screens should not contain database logic; Flutter services call FastAPI, FastAPI services contain business rules, and SQLAlchemy models represent persistence.

5. Backend Code – Core Infrastructure

5.1 app/main.py

Purpose: Application entry point. Creates the FastAPI application, configures middleware, registers routes, and exposes the service through Uvicorn.

Responsibilities:

- Create FastAPI app instance.
- Register authentication, attendance, admin, fees, salary and report routers.
- Configure CORS for Flutter Web during development.
- Expose health/test endpoints when required.
- Avoid mixing business logic into the application entry point.

5.2 app/core/config.py

Reads environment variables from **.env** and exposes application settings such as database URL, JWT secret, JWT algorithm and token expiration.

```
APP_NAME=NOVOX
APP_ENV=development
DATABASE_URL=mysql+pymysql://root:YOUR_PASSWORD@localhost:3306/novox_db
JWT_SECRET_KEY=CHANGE_THIS_TO_A_LONG_RANDOM_SECRET
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
```

5.3 app/database/connection.py

Creates the SQLAlchemy engine and SessionLocal factory. Every API request that needs the database receives a session through the dependency layer.

```
engine = create_engine(settings.DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)
```

5.4 app/database/base.py

Defines the common SQLAlchemy DeclarativeBase from which all ORM models inherit. This gives SQLAlchemy a common metadata registry.

5.5 app/core/dependencies.py

Provides the get_db dependency. It opens a database session before a route runs and guarantees that the session is closed afterward.

5.6 app/core/timezone.py

Provides India-time helpers using **Asia/Kolkata**. Attendance calculations should use the server's India time helper instead of trusting a client-supplied clock.

5.7 app/core/security.py

Contains password hashing and JWT functions. Passwords are stored as hashes; the plain password should never be stored in MySQL.

5.8 app/core/websocket_manager.py

Maintains active WebSocket connections and broadcasts JSON messages to connected admin clients. It also removes connections that have disconnected or failed.

6. Backend Code – Authentication & Roles

6.1 User model

The User table is the central identity record. It stores name, email, password hash, role, phone, active status and creation time.

Field	Meaning
id	Primary user identifier
name	Display name
email	Unique login identifier
password_hash	Secure password hash
role	student / staff / admin
phone	Optional contact number
is_active	Account access control
created_at	Account creation timestamp

6.2 JWT flow

After successful login, FastAPI creates a JWT containing the user identity and expiration time. Flutter stores it securely. Every protected API request sends it as **Authorization: Bearer <token>**.

6.3 Role dependencies

The role dependency first authenticates the JWT, retrieves the User, checks that the account is active, and then checks whether the user's role is allowed for the route.

Role	Main access
Student	Own dashboard, attendance, fees and payment history
Staff	Own profile, attendance and salary information
Admin	Central monitoring, student/staff management, fees, salary, reports and geofence configuration

6.4 Test accounts

Role	Email	Password	Reference ID
Admin	admin@novox.com	Admin@123	NOVOX-ADM-001
Staff	staff@novox.com	Staff@123	NOVOX-STF-001
Student	student@novox.com	Student@123	NOVOX-STU-001

Development only: These credentials are test credentials. Production credentials should be changed and managed securely.

7. Database Models & Relationships

7.1 Identity relationships

```
User
 ■■■ 1:1 ■■ Student
 ■■■ 1:1 ■■ Staff
```

A User represents authentication. Student and Staff represent role-specific profile information.

7.2 Attendance relationships

```
User
  |
  +---- Attendance ----< AttendanceBreak

GeofenceSettings
  |
  +---- used by attendance distance validation
```

7.3 Fees relationships

```
FeeStructure
      |
      +---- StudentFee ----< Payment
                    |
                    +---- Student
```

7.4 Salary relationships

```
Staff
  |
  +---- SalaryStructure
  |
  +---- SalarySlip
```

Model	Important fields / purpose
GeofenceSettings	Latitude, longitude, radius, active state
Attendance	User, date, check-in, checkout, status, GPS, breaks, work minutes
AttendanceBreak	Break start/end and duration
FeeStructure	Course, fee type, total amount, description
StudentFee	Total, paid, pending and status for a student
Payment	Payment amount, method, transaction, date, receipt
SalaryStructure	Basic, allowance, deduction, effective date
SalarySlip	Month/year snapshot and calculated net salary

8. Attendance Module – Detailed Code Explanation

8.1 attendance_service.py

This service contains the core attendance rules. It is intentionally separated from the route so the route remains thin and the business logic can be reused or tested.

check_in_user

- Find today's attendance for the authenticated user.
- Reject duplicate check-in.
- Load active geofence settings.
- Calculate distance between device coordinates and campus coordinates.
- Reject the request when outside the configured radius.
- Use the server's India timestamp.
- Compare the timestamp to 10:00 AM.
- Set PRESENT / ON_TIME when the time is at or before the cutoff.
- Set PRESENT / LATE when the time is after the cutoff.
- Persist latitude, longitude and calculated distance.

start_break / end_break

Break records are stored separately. This makes it possible to calculate total break duration rather than relying on a single flag.

checkout_user

Checkout uses a server timestamp, requires that there is no active break, and calculates work minutes as elapsed time minus recorded break minutes.

auto_checkout_user

The planned automatic checkout behavior is triggered when the system detects that the user has exited the geofence. The service closes the attendance and calculates work time.

8.2 attendance routes

Endpoint	Purpose
POST /attendance/check-in	GPS-validated check-in
POST /attendance/break/start	Start break
POST /attendance/break/end	End break
POST /attendance/checkout	Manual checkout
POST /attendance/auto-checkout	Automatic checkout workflow
GET /attendance/history	Attendance history with optional month/year

8.3 Flutter AttendanceService

The Flutter AttendanceService sends coordinates to the API, converts JSON responses into AttendanceModel objects, starts/ends breaks, checks out, and loads history.

9. Geofencing – Detailed Explanation

The original specification states that the device GPS coordinates are compared against the institution perimeter. Inside the boundary enables check-in; outside the boundary locks the action and displays an 'Out of Campus' state. ■filecite■turn35file1■

9.1 GeofenceSettings

Field	Purpose
latitude	Campus center latitude
longitude	Campus center longitude
radius_meters	Allowed attendance radius
active	Whether the configuration is enabled
created_at / updated_at	Configuration audit timestamps

9.2 Calculation concept

```
distance = distance(device_lat, device_lon,
                    campus_lat, campus_lon)

if distance <= radius_meters:
    ALLOW
else:
    DENY
```

9.3 Why validation must happen on the server

The Flutter client can improve the user experience by showing distance and enabling/disabling buttons, but the backend must repeat the validation. This prevents a modified client from bypassing the campus boundary.

9.4 Admin Geofence page

AdminGeofenceScreen loads the current configuration, displays latitude/longitude/radius, validates input, and calls the create/update API. This page controls the institution's attendance perimeter.

10. Fees Module – Detailed Explanation

10.1 Backend fee models

FeeStructure defines what a fee is. StudentFee represents what a specific student owes. Payment represents each payment transaction.

10.2 Fee status calculation

```
paid_amount >= total_amount
        -> PAID

paid_amount > 0 and paid_amount < total_amount
        -> PARTIALLY_PAID

paid_amount == 0
        -> PENDING
```

10.3 Payment rules

- Do not allow payment greater than the outstanding amount.
- Update paid amount after successful payment.
- Recalculate pending amount.
- Recalculate status.
- Generate a unique receipt number.
- Store payment date, method and optional transaction reference.

10.4 Important API pages

Endpoint	Purpose
POST /fees/structure	Create fee structure
POST /fees/student	Assign fee to student
POST /fees/payment	Record payment
GET /fees/pending	Admin pending fees
GET /fees/my-fees	Student's own fees
GET /fees/my-payment-history	Student payment history
GET /fees/receipt/{receipt}/pdf	Receipt PDF

10.5 Flutter fee pages

StudentFees displays assigned fees, paid/pending amounts and payment history. AdminFeesScreen provides fee structure creation, pending fee monitoring and payment recording. Receipt PDF downloads are handled through Dio as byte responses.

11. Salary Module – Detailed Explanation

11.1 SalaryStructure

Stores the active pay configuration for staff: basic salary, allowance, deduction and effective date.

11.2 SalarySlip

A salary slip is a monthly snapshot. This is important because future salary structure changes should not silently rewrite historical payslips.

11.3 Net salary calculation

```
net_salary = basic_salary + allowance - deduction
```

11.4 Salary APIs

Endpoint	Purpose
POST /salary/structure	Create/update salary configuration
GET /salary/staff/{staff_id}	Admin view of staff salary data
POST /salary/slip/{staff_id}/{year}/{month}	Generate monthly slip
GET /salary/my-slips	Staff's own slips
GET /salary/slip/{slip_id}/pdf	Salary slip PDF

11.5 Flutter salary pages

AdminSalaryScreen configures staff salary and generates slips. StaffSalaryScreen lists the authenticated staff member's monthly salary slips and displays itemized values.

12. Admin Module – Every Major Page

12.1 AdminDashboard

The central operational dashboard. It loads summary statistics, connects to the admin WebSocket, and navigates to management/report pages.

Dashboard area	Meaning
Total Students	Active student population
Total Staff	Active staff population
Present Today	Attendance records marked present
Late Today	Attendance records after the 10:00 AM cutoff
Checked Out	Records with checkout time
Pending Fee Students	Students with outstanding fees
Total Pending Amount	Outstanding monetary balance

12.2 AdminAttendance

Displays today's attendance records, including arrival status, check-in, checkout, attendance status and check-in distance.

12.3 AdminStudents

Provides student listing, creation and deactivation. The current page collects full name, email, password, student code, course, batch and phone. ■filecite■turn34file0■L165-L187■ It then calls AdminService.createStudent and refreshes the list. ■filecite■turn34file0■L482-L529■

12.4 AdminStaff

Provides staff creation, editing, deactivation and navigation to salary configuration.

12.5 AdminFeesScreen

Provides pending fee monitoring, collected/pending totals, payment recording and fee structure creation.

12.6 AdminGeofenceScreen

Creates or updates the campus latitude, longitude and radius used by attendance validation.

12.7 AdminSalaryScreen

Loads a selected staff member's salary data, configures basic/allowance/deduction and generates monthly slips.

13. Admin Reports – Detailed Explanation

13.1 AdminReportsScreen

Acts as the report navigation hub. It provides access to summary, attendance, late-entry, fee and salary reports.

13.2 AdminAttendanceReportScreen

Loads attendance report data for a selected date range, displays status and check-in information, and provides refresh/clear-filter controls.

13.3 AdminLateEntryReportScreen

Filters late-entry records by date range and displays late check-in information including time and geofence distance.

13.4 Fee report

Shows fee collection, pending balances and payment information for administration.

13.5 Salary report

Provides salary-related reporting for administrative review.

13.6 Export endpoints

Export	Endpoint
Attendance PDF	/admin/reports/export/attendance/pdf
Attendance CSV	/admin/reports/export/attendance/csv
Fee PDF	/admin/reports/export/fees/pdf
Salary PDF	/admin/reports/export/salary/pdf

The Flutter ReportService requests these exports as bytes. A future/complete download layer should save those bytes using path_provider on supported platforms.

14. Flutter Core Infrastructure

14.1 main.dart

Application entry point. It initializes Flutter, configures the root MaterialApp, and starts the authentication/navigation flow.

14.2 api_constants.dart

Centralizes all API paths and the backend base URL. When running Flutter Web in Edge on the same development PC, the current development base URL is **http://127.0.0.1:8000**. For an Android emulator, the base URL would normally be **http://10.0.2.2:8000**; these are environment-specific settings and should not be mixed.

14.3 api_client.dart

Creates a Dio instance with the base URL, timeouts and JSON headers. Its interceptor loads the JWT from SecureStorage and adds the Bearer token to outgoing requests.

```
onRequest:
    token = await SecureStorage.getToken()

    if token exists:
        Authorization = Bearer <token>

    continue request
```

14.4 secure_storage.dart

Stores and retrieves authentication information using flutter_secure_storage. The current project uses static methods such as getToken(), getRole(), getName() and clear().

14.5 Providers

Providers are the intended state-management layer for shared UI state. Services should remain responsible for API communication while providers can coordinate loading/error/data state between screens.

15. Flutter Models & Services – Page-by-Page Responsibilities

15.1 Models

Model	Purpose
AttendanceModel	Represents a user's attendance record
AdminDashboardModel	Represents admin summary metrics
StudentAdminModel	Represents an admin-facing student record
StaffAdminModel	Represents an admin-facing staff record
StaffModel	Represents authenticated staff profile
FeeModel	Represents student fee information
StudentPaymentModel	Represents student payment history
SalaryModel	Represents staff salary slip
AdminSalaryModel	Represents admin staff salary data
GeofenceModel	Represents geofence configuration

15.2 Services

Service	Responsibilities
AttendanceService	Check-in, break, checkout, today and history
AdminService	Admin dashboard, students, staff, salary, fees, geofence
FeeService	Student fee list, payment history and receipt PDF
SalaryService	Staff salary slip list
StaffService	Staff profile
ReportService	Reports and exports
AdminWebSocketService	Admin live attendance stream

15.3 Service design rule

A Flutter screen should call a service rather than directly constructing HTTP requests. This keeps UI code readable and allows API behavior to be changed centrally.

16. Student Portal – Page-by-Page

16.1 StudentDashboard

The student dashboard is the main landing page. It can show attendance state, fees and navigation to attendance/fee screens. It obtains attendance through AttendanceService and fees through FeeService.

16.2 StudentAttendanceScreen

Shows today's attendance and historical attendance. The current naming convention uses StudentAttendanceScreen to match navigation references from StudentDashboard.

16.3 StudentFees

Displays fee obligations, payment status and payment history. It can also initiate secure receipt PDF retrieval for the authenticated student.

16.4 Student flow

```
LOGIN
  ↓
Student Dashboard
  ■■■ Attendance
  ■     ■■■ GPS permission
  ■     ■■■ Check-in
  ■     ■■■ Break
  ■     ■■■ Checkout
  ■     ■■■ History
  ■
  ■■■ Fees
        ■■■ Current fees
        ■■■ Paid / Pending
        ■■■ Payment history
        ■■■ Receipt PDF
```

The source specification defines the student workflow as login, GPS geofence validation, time evaluation, break and checkout tracking, attendance history/profile/fees. ■filecite■turn35file0■

17. Staff Portal – Page-by-Page

17.1 StaffDashboard

Main staff landing page. It loads the authenticated staff profile and provides navigation to profile, attendance and salary.

17.2 StaffProfileScreen

Displays staff identity, employee code, designation, department, contact information and joining information.

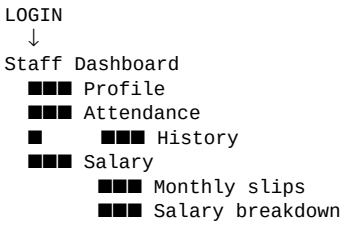
17.3 StaffSalaryScreen

Lists monthly salary slips and displays basic salary, allowance, deduction and net salary.

17.4 Staff attendance

Staff use the same attendance service rules as students for daily attendance where the backend route permits the staff role.

17.5 Staff flow



This matches the source specification's staff role: dashboard/profile, daily attendance, monthly salary slips and salary breakdown details. ■filecite■turn35file0■

18. Real-Time WebSocket Architecture

18.1 Why WebSocket is used

REST is suitable for loading a dashboard, but an admin dashboard that should reflect attendance changes immediately benefits from a persistent WebSocket connection.

18.2 Backend

The backend WebSocket endpoint is **/ws/admin**. It receives a JWT token, verifies that the user is an active admin, then registers the WebSocket with ConnectionManager.

18.3 Flutter

AdminWebSocketService retrieves the token from SecureStorage, creates a WebSocket URL from the configured HTTP base URL, connects to /ws/admin, decodes JSON messages and returns a typed stream.

```
http://127.0.0.1:8000
        |
        +--> ws://127.0.0.1:8000/ws/admin?token=<JWT>
```

18.4 AdminDashboard lifecycle

- Connect after the dashboard loads.
- Listen for attendance events.
- Update/reload relevant dashboard state.
- Reconnect after a connection failure when appropriate.
- Cancel the stream subscription and close the socket in dispose().

18.5 Security note

The current development design passes the token as a query parameter. Query-string tokens can appear in server/proxy logs, so production deployments should consider a safer authentication mechanism for WebSockets.

19. API Catalog

Module	Important endpoints
Auth	POST /auth/login
Student	GET /student/dashboard
Staff	GET /staff/dashboard, GET /staff/profile
Attendance	POST /attendance/check-in, /break/start, /break/end, /checkout; GET /attendance/history
Fees	GET /fees/my-fees, /my-payment-history; POST /fees/payment; admin fee endpoints
Salary	GET /salary/my-slips; admin salary structure/slip endpoints
Admin	GET /admin/dashboard, /admin/attendance/today, /admin/fees/pending
Management	Student/staff create, list, update/deactivate
Geofence	GET/POST/PUT /admin/geofence
Reports	Summary, attendance, late-entry, fees, salary
Exports	Attendance PDF/CSV, Fee PDF, Salary PDF
WebSocket	WS /ws/admin

Recommended API discipline

- Validate all incoming data server-side.
- Return consistent JSON structures.
- Use HTTP status codes consistently.
- Never trust client timestamps for attendance.
- Never trust client role claims; derive role from the authenticated database user.
- Apply ownership checks to student-specific fee/receipt data.
- Use Decimal/Numeric for money.

20. MySQL Database Architecture

20.1 Main tables

Table	Key purpose
users	Authentication and role
students	Student profile
staff	Staff profile
attendance	Daily attendance
attendance_breaks	Break sessions
geofence_settings	Campus perimeter
fee_structures	Course/fee definitions
student_fees	Student fee obligations
payments	Payment transactions and receipts
salary_structures	Current staff salary configuration
salary_slips	Monthly salary snapshots

20.2 Money types

Fee and salary monetary values should use SQL Numeric(12,2), not floating-point fields. This avoids common binary floating-point precision problems in financial calculations.

20.3 Migration strategy

Once Alembic migrations are used, database schema changes should be handled through migration files rather than repeatedly calling Base.metadata.create_all() in application startup.

21. Security Architecture

21.1 Authentication

- Passwords are hashed using a password hashing library; the original password is not stored.
- JWT tokens are signed using a server-side secret.
- Protected routes require a valid, non-expired token.

21.2 Authorization

- Role is checked server-side for every protected functional area.
- Student and staff data should be scoped to the authenticated user.
- Admin routes require the admin role.

21.3 Attendance integrity

- Server-side timestamping prevents the client from choosing an arbitrary attendance time.
- Server-side geofence validation prevents a modified client from simply enabling check-in.
- Attendance records store coordinates and calculated distance for auditing.

21.4 Fee/receipt security

A student receipt endpoint must verify that the authenticated student owns the underlying payment/receipt before returning the PDF. Admins can access authorized administrative receipt functions.

21.5 Production security checklist

- Use HTTPS.
- Use a strong random JWT secret.
- Use production database credentials outside source control.
- Restrict CORS to actual frontend origins.
- Avoid query-string JWTs for production WebSockets where possible.
- Add audit logging for administrative changes.
- Back up MySQL.
- Rotate test credentials before deployment.

22. eSSL Biometric Integration – Planned Extension

The eSSL integration discussed for NOVOX is a planned extension, not a component that should be treated as already implemented in the current codebase.

22.1 Target architecture

```
eSSL Biometric Device
        |
        v
eSSL middleware / integration API
        |
        v
NOVOX FastAPI integration layer
        |
        v
Attendance matching service
        |
        +---- MySQL attendance
        |
        +---- WebSocket event
        |
        v
Admin Dashboard
```

22.2 Attendance source

A useful design is to record the attendance source, for example **MOBILE** or **ESSL**. The same attendance table can then support both mobile GPS attendance and biometric attendance while preserving how the record was created.

22.3 Identity matching

Each eSSL employee/device identifier should be mapped to a NOVOX student_code or employee_code. The exact API, middleware and synchronization behavior depends on the specific eSSL device/model and integration software available.

22.4 Recommended integration sequence

- Identify the exact eSSL model.
- Confirm whether the installation exposes eBioServer/eTimeTrackLite or another supported integration interface.
- Map eSSL employee IDs to NOVOX student/staff codes.
- Create an integration service in FastAPI.
- Normalize device punch logs into NOVOX attendance events.
- Apply duplicate protection and timezone normalization.
- Broadcast new attendance events to the Admin WebSocket.
- Add an attendance_source field for auditability.

23. Development & Run Architecture

23.1 Backend

```
cd D:\NOVOX\NOVOX-EDTECH\backend
venv\Scripts\activate
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

The current development server has been successfully started with Uvicorn on port 8000.

23.2 Flutter Web / Edge

For Flutter Web running in Microsoft Edge on the same PC as FastAPI, the development API base URL should use 127.0.0.1:8000.

```
static const String baseUrl = "http://127.0.0.1:8000";
```

23.3 Android emulator

For an Android emulator, the conventional host alias is 10.0.2.2. This is a different environment from Edge.

```
static const String baseUrl = "http://10.0.2.2:8000";
```

23.4 Physical phone

For a physical phone, use the development PC's LAN IPv4 address and ensure the phone and PC are on the same network. Windows Firewall may also need a development exception for port 8000.

24. Troubleshooting Guide

Problem	Likely cause	Check
Dio connection timeout in Edge	Wrong API base URL or backend unreachable	Use 127.0.0.1:8000 and test /docs
Android cannot connect	Wrong host alias	Use 10.0.2.2 for emulator or LAN IP for phone
CORS error in Edge	Backend CORS policy	Configure FastAPI CORSMiddleware
401 Unauthorized	Missing/expired JWT	Check SecureStorage and Authorization header
403 Forbidden	Wrong role	Check JWT user role and route dependency
Student model compile error	Duplicate/mismatched model definitions	Keep StudentAdminModel in one model file with email/phone
WebSocket compile error	Nullable stream or missing JSON import	Await connect() and use dart:convert
MySQL connection error	Invalid DATABASE_URL / credentials	Check .env and MySQL service
Duplicate YAML mapping	Repeated dependencies section	Keep one dependencies section in pubspec.yaml

Debugging order

1. Is MySQL running?
2. Can FastAPI start without startup errors?
3. Can Edge open http://127.0.0.1:8000/docs?
4. Does POST /auth/login work in Swagger?
5. Does Flutter use the correct base URL?
6. Does Flutter send Authorization: Bearer <token>?
7. Is the user's role correct?
8. Does the protected endpoint work in Swagger?
9. Only then debug the Flutter screen.

25. Code Page / File Map

Code page/file	What it controls
backend/app/main.py	FastAPI application and route registration
backend/app/core/config.py	Environment configuration
backend/app/core/security.py	Password hashing and JWT
backend/app/core/dependencies.py	DB session and auth dependencies
backend/app/core/timezone.py	India server time
backend/app/core/websocket_manager.py	Live admin connections
backend/app/database/connection.py	MySQL/SQLAlchemy engine
backend/app/database/base.py	ORM base class
backend/app/models/*	Database entities
backend/app/services/attendance_service.py	Attendance business rules
backend/app/services/fee_service.py	Fees/payment rules
backend/app/services/salary_service.py	Salary rules
backend/app/services/report_service.py	Report generation/data
backend/app/routes/*	HTTP/WebSocket endpoints
mobile/lib/main.dart	Flutter application entry
mobile/lib/core/constants/api_constants.dart	API paths/base URL
mobile/lib/core/network/api_client.dart	Dio client/auth headers
mobile/lib/core/storage/secure_storage.dart	JWT/token storage
mobile/lib/models/*	JSON-to-Dart data objects
mobile/lib/services/*	REST/WebSocket communication
mobile/lib/screens/auth/*	Login/authentication UI
mobile/lib/screens/student/*	Student portal
mobile/lib/screens/staff/*	Staff portal
mobile/lib/screens/admin/*	Admin portal
mobile/lib/widgets/*	Reusable UI components

26. Important Sequence Diagrams

26.1 Login

```
User -> Flutter Login
Flutter -> FastAPI: POST /auth/login
FastAPI -> MySQL: Find user
FastAPI -> Security: Verify password
Security -> FastAPI: Valid
FastAPI -> Flutter: JWT + role
Flutter -> SecureStorage: Save token
Flutter -> Dashboard: Navigate by role
```

26.2 Check-in

```
User -> Flutter: Tap Check-In
Flutter -> Geolocator: Get GPS
Geolocator -> Flutter: latitude/longitude
Flutter -> FastAPI: POST /attendance/check-in
FastAPI -> MySQL: Load geofence
FastAPI: Calculate distance + server time
FastAPI -> MySQL: Save attendance
FastAPI -> Admin WebSocket: Broadcast event
FastAPI -> Flutter: Attendance result
```

26.3 Fee payment

```
Admin -> Flutter: Record payment
Flutter -> FastAPI: POST /fees/payment
FastAPI -> MySQL: Load StudentFee
FastAPI: Validate amount
FastAPI -> MySQL: Save Payment + update fee
FastAPI: Generate receipt number
FastAPI -> Flutter: Payment result
Flutter -> Admin: Refresh pending/collected totals
```

27. Recommended Next Development Phases

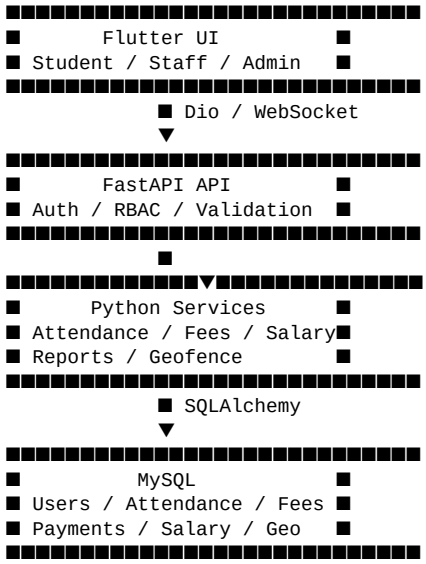
Phase	Work
1	Finish clean compilation across Flutter Web
2	Verify login for Admin / Staff / Student
3	Verify MySQL schema and seed/test accounts
4	Test GPS geofence with a real location
5	Test attendance cutoff, break and checkout
6	Test fee assignment, payment and receipt PDF
7	Test salary configuration and salary slip PDF
8	Test admin reports and exports
9	Test WebSocket live attendance
10	Add production CORS, HTTPS and security hardening
11	Identify eSSL device and implement biometric integration
12	Deployment and backup strategy

Definition of a complete MVP

- All three roles can log in.
- Role-based dashboard routing works.
- Student/staff attendance works with server time and geofence.
- Admin sees attendance live and historically.
- Fees and receipts work.
- Salary structures and slips work.
- Reports export successfully.
- Database constraints prevent duplicate/invalid records.
- Unauthorized users cannot access other users' protected data.

28. Final Architecture Summary

NOVOX is designed as a layered system. Flutter is the presentation layer, Dio-based services are the client integration layer, FastAPI exposes secure REST/WebSocket endpoints, Python services contain business rules, SQLAlchemy manages persistence, and MySQL stores the system of record.




```
Future biometric input:
eSSL → integration layer → FastAPI → MySQL
```

The original NOVOX specification's central principles are preserved: role-based access, GPS geofence validation, a 10:00 AM attendance cutoff, break/checkout tracking, fees and receipts, staff salary management, server-side timestamps, and synchronization to the admin dashboard. ■filecite■turn35file0■ ■filecite■turn35file1■

Important implementation note: This document explains the architecture and code responsibilities established during the project work. It is not a line-by-line copy of every source file, and planned integrations such as eSSL are clearly identified as future work.

END OF DOCUMENT